

```
# Extended Individual Support Module
## Machine Learning via Large Number Theory & Markov Chains
```

```
---
```

```
## Extended Module Architecture
```

```
```python
```

```
individual_support_ml_extended.py
Machine learning extensions using large number theory and Markov chains
for learning individual system dynamics from population data
"""
```

```
import numpy as np
from scipy import stats, linalg
from scipy.special import logsumexp
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple, Callable
from collections import defaultdict
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')
```

```
Import base module components
from individual_support_module import (
 SystemState, ControlInput, PhysicsMapping,
 SystemDynamics, Controller, Observer, ReferenceFrame
)
```

```
=====
MARKOV CHAIN STATE SPACE
=====
```

```
@dataclass
class DiscreteState:
 """
 Discretized state for Markov chain analysis
 Maps continuous state to discrete bins
 """
 pressure_bin: int # 0-9 (10 bins)
 margin_bin: int # 0-9
 temperature_bin: int # 0-9
 drift_bin: int # 0-9

 def to_index(self) -> int:
 """Convert multi-dimensional state to single index"""
 return (
 self.pressure_bin * 1000 +
 self.margin_bin * 100 +
 self.temperature_bin * 10 +
 self.drift_bin
)
```

```
@classmethod
```

```

def from_index(cls, idx: int) -> 'DiscreteState':
 """Recover multi-dimensional state from index"""
 return cls(
 pressure_bin=(idx // 1000) % 10,
 margin_bin=(idx // 100) % 10,
 temperature_bin=(idx // 10) % 10,
 drift_bin=idx % 10
)

@classmethod
def from_continuous(cls, state: SystemState) -> 'DiscreteState':
 """Discretize continuous state"""
 return cls(
 pressure_bin=min(9, int(state.pressure)),
 margin_bin=min(9, int(state.margin)),
 temperature_bin=min(9, int(state.temperature)),
 drift_bin=min(9, int((state.drift + 5) * 0.9)) # Map [-5,5] to [0,9]
)

```

```

class MarkovChainModel:
 """

```

Markov chain model of state transitions  
 $P(s_{t+1} \mid s_t, a_t)$

Uses large number theory: as we observe more transitions,  
 empirical transition probabilities converge to true probabilities  
 """

```

def __init__(self, n_states: int = 10000, n_actions: int = 100):
 """
 n_states: number of discrete states (10^4 for 4D discretization)
 n_actions: number of discrete action types
 """
 self.n_states = n_states
 self.n_actions = n_actions

 # Transition counts: C(s, a, s')
 self.transition_counts = np.zeros((n_states, n_actions, n_states))

 # Prior (Dirichlet) for Bayesian smoothing
 self.prior_alpha = 0.1

 # Transition probabilities (learned)
 self.transition_probs = None

 # Reward model: R(s, a)
 self.reward_model = np.zeros((n_states, n_actions))
 self.reward_counts = np.zeros((n_states, n_actions))

 # Ergodic properties
 self.stationary_distribution = None
 self.mixing_time = None

def observe_transition(
 self,
 state: DiscreteState,

```

```

 action: int,
 next_state: DiscreteState,
 reward: float
):
 """
 Observe a state transition
 Law of Large Numbers: empirical mean $\rightarrow$ true mean as $N \rightarrow \infty$
 """
 s = state.to_index()
 s_next = next_state.to_index()

 # Update transition counts
 self.transition_counts[s, action, s_next] += 1

 # Update reward model (running average)
 self.reward_counts[s, action] += 1
 n = self.reward_counts[s, action]
 self.reward_model[s, action] += (reward - self.reward_model[s, action]) / n

def fit(self):
 """
 Compute transition probabilities from counts
 Uses Bayesian smoothing (MAP estimate with Dirichlet prior)
 """
 # Add prior counts (Laplace smoothing)
 smoothed_counts = self.transition_counts + self.prior_alpha

 # Normalize to get probabilities
 # $P(s' | s, a) = C(s, a, s') / \sum_{s'} C(s, a, s')$
 row_sums = smoothed_counts.sum(axis=2, keepdims=True)
 self.transition_probs = smoothed_counts / np.maximum(row_sums, 1e-10)

 return self

def predict_next_state_distribution(
 self,
 state: DiscreteState,
 action: int
) -> np.ndarray:
 """
 Get probability distribution over next states
 $P(s_{t+1} | s_t, a_t)$
 """
 if self.transition_probs is None:
 self.fit()

 s = state.to_index()
 return self.transition_probs[s, action, :]

def sample_next_state(
 self,
 state: DiscreteState,
 action: int
) -> DiscreteState:
 """Sample next state from learned distribution"""
 dist = self.predict_next_state_distribution(state, action)
 next_idx = np.random.choice(self.n_states, p=dist)

```

```

return DiscreteState.from_index(next_idx)

def compute_stationary_distribution(self, action: int = 0) -> np.ndarray:
 """
 Compute stationary distribution π where $\pi = \pi P$
 Uses power iteration (ergodic theorem)

 For ergodic chains: temporal average $\rightarrow$ ensemble average as $T \rightarrow \infty$
 """
 if self.transition_probs is None:
 self.fit()

 # Transition matrix for given action
 P = self.transition_probs[:, action, :]

 # Power iteration
 pi = np.ones(self.n_states) / self.n_states
 for _ in range(1000):
 pi_new = pi @ P
 if np.allclose(pi, pi_new, atol=1e-6):
 break
 pi = pi_new

 self.stationary_distribution = pi
 return pi

def estimate_mixing_time(self, action: int = 0, epsilon: float = 0.01) -> int:
 """
 Estimate mixing time: how long to reach stationary distribution
 Uses total variation distance
 """
 if self.transition_probs is None:
 self.fit()

 P = self.transition_probs[:, action, :]
 pi_stat = self.compute_stationary_distribution(action)

 # Start from arbitrary initial distribution
 pi_t = np.zeros(self.n_states)
 pi_t[0] = 1.0 # Start at state 0

 for t in range(1, 1000):
 pi_t = pi_t @ P
 tv_distance = 0.5 * np.sum(np.abs(pi_t - pi_stat))

 if tv_distance < epsilon:
 self.mixing_time = t
 return t

 self.mixing_time = 1000
 return 1000

def compute_entropy_rate(self, action: int = 0) -> float:
 """
 Compute entropy rate of the Markov chain
 $H = -\sum_s \pi(s) \sum_{s'} P(s'|s) \log P(s'|s)$
 """

```

```

Measures unpredictability of system dynamics
"""
if self.stationary_distribution is None:
 self.compute_stationary_distribution(action)

P = self.transition_probs[:, action, :]
pi = self.stationary_distribution

Avoid log(0)
P_safe = np.maximum(P, 1e-10)

$H = -\sum_s \pi(s) \sum_{s'} P(s'|s) \log P(s'|s)$
entropy = -np.sum(pi[:, None] * P * np.log(P_safe))

return entropy

=====
POPULATION-LEVEL LEARNING
=====

@dataclass
class PopulationData:
 """
 Collection of trajectories from multiple individuals
 Law of Large Numbers: population statistics → true parameters
 """
 trajectories: List[List[Tuple[SystemState, ControlInput, float]]] = field(
 default_factory=list
)
 n_individuals: int = 0
 total_observations: int = 0

 def add_trajectory(
 self,
 trajectory: List[Tuple[SystemState, ControlInput, float]]
):
 """Add trajectory from one individual"""
 self.trajectories.append(trajectory)
 self.n_individuals += 1
 self.total_observations += len(trajectory)

 def get_empirical_statistics(self) -> Dict[str, float]:
 """
 Compute population-level statistics
 Central Limit Theorem: sample means are approximately normal
 """
 all_pressures = []
 all_margins = []
 all_temperatures = []
 all_drifts = []
 all_strains = []

 for traj in self.trajectories:
 for state, _, _ in traj:
 all_pressures.append(state.pressure)
 all_margins.append(state.margin)

```

```

all_temperatures.append(state.temperature)
all_drifts.append(state.drift)
all_strains.append(state.strain)

```

```

return {
 'mean_pressure': np.mean(all_pressures),
 'std_pressure': np.std(all_pressures),
 'mean_margin': np.mean(all_margins),
 'std_margin': np.std(all_margins),
 'mean_temperature': np.mean(all_temperatures),
 'std_temperature': np.std(all_temperatures),
 'mean_drift': np.mean(all_drifts),
 'std_drift': np.std(all_drifts),
 'mean_strain': np.mean(all_strains),
 'std_strain': np.std(all_strains),
 'n_observations': len(all_pressures)
}

```

```

def estimate_confidence_intervals(
 self,
 confidence: float = 0.95
) -> Dict[str, Tuple[float, float]]:
 """
 Compute confidence intervals using CLT
 As $N \rightarrow \infty$, sample mean $\pm z^*(\sigma/\sqrt{N})$ contains true mean with probability $\approx$ confidence
 """
 stats = self.get_empirical_statistics()
 n = stats['n_observations']
 z_score = stats.norm.ppf((1 + confidence) / 2)

 intervals = {}
 for key in ['pressure', 'margin', 'temperature', 'drift', 'strain']:
 mean = stats[f'mean_{key}']
 std = stats[f'std_{key}']
 margin_of_error = z_score * std / np.sqrt(n)
 intervals[key] = (mean - margin_of_error, mean + margin_of_error)

 return intervals

```

```

class PopulationLearner:
 """

```

Learn universal dynamics from population data

Key theorem: As population size  $\rightarrow \infty$ , empirical distributions converge to true underlying distribution (Glivenko-Cantelli)

```

 """

```

```

def __init__(self):
 self.population_data = PopulationData()
 self.markov_model = MarkovChainModel()

```

```

 # Learned population-level parameters
 self.population_dynamics = None
 self.intervention_effectiveness = {}

```

```

 # Clustering: different "types" of individuals

```

```

self.n_clusters = 5
self.cluster_models = {}

def ingest_individual_trajectory(
 self,
 trajectory: List[Tuple[SystemState, ControlInput, float]],
 individual_id: str
):
 """
 Ingest trajectory from one individual
 Each transition updates our Markov chain model
 """
 self.population_data.add_trajectory(trajectory)

 # Convert to discrete states and actions
 for i in range(len(trajectory) - 1):
 state, action, reward = trajectory[i]
 next_state, _, _ = trajectory[i + 1]

 discrete_state = DiscreteState.from_continuous(state)
 discrete_next = DiscreteState.from_continuous(next_state)

 # Discretize action (simplified: map control to action index)
 action_idx = self._discretize_action(action)

 # Observe transition
 self.markov_model.observe_transition(
 discrete_state,
 action_idx,
 discrete_next,
 reward
)

def _discretize_action(self, control: ControlInput) -> int:
 """
 Map continuous control input to discrete action space
 Simple quantization scheme
 """
 # Create action signature
 signature = (
 int(control.pressure_relief * 10),
 int(control.capacity_building * 10),
 int(control.cooling_input * 10),
 int(control.realignment_force * 10 + 50)
)

 # Map to single index (simplified)
 return min(99, hash(signature) % 100)

def fit_population_model(self):
 """
 Fit Markov chain model from all observed data
 LLN guarantees convergence as $N \rightarrow \infty$
 """
 print(" Fitting population model from data...")
 print(f" Individuals: {self.population_data.n_individuals}")
 print(f" Total transitions: {self.population_data.total_observations}")

```

```

Fit Markov chain
self.markov_model.fit()

Compute ergodic properties
print("\n Computing ergodic properties...")
self.markov_model.compute_stationary_distribution(action=0)
mixing_time = self.markov_model.estimate_mixing_time(action=0)
entropy_rate = self.markov_model.compute_entropy_rate(action=0)

print(f" Mixing time: {mixing_time} steps")
print(f" Entropy rate: {entropy_rate:.3f} nats/step")

Estimate intervention effectiveness
self._estimate_intervention_effects()

return self

def _estimate_intervention_effects(self):
 """
 Estimate causal effect of interventions using population data
 Effect = E[strain | intervention] - E[strain | no intervention]
 """
 print("\n Estimating intervention effectiveness...")

 for action_idx in range(min(10, self.markov_model.n_actions)):
 # Compute expected reward for this action across all states
 # Weight by stationary distribution
 pi = self.markov_model.stationary_distribution
 expected_reward = np.sum(
 pi * self.markov_model.reward_model[:, action_idx]
)

 self.intervention_effectiveness[action_idx] = expected_reward

 # Sort by effectiveness
 sorted_actions = sorted(
 self.intervention_effectiveness.items(),
 key=lambda x: x[1],
 reverse=True
)

 print(f" Top 3 most effective actions:")
 for action, effect in sorted_actions[:3]:
 print(f" Action {action}: {effect:.3f}")

def cluster_individuals(self):
 """
 Cluster individuals into archetypes using k-means
 Different clusters may have different dynamics
 """
 print(f"\n Clustering individuals into {self.n_clusters} archetypes...")

 # Extract features from each trajectory
 features = []
 for traj in self.population_data.trajectories:
 # Compute summary statistics for this individual

```



```

pressures = [s.pressure for s, _, _ in traj]
margins = [s.margin for s, _, _ in traj]
temps = [s.temperature for s, _, _ in traj]

feature_vec = [
 np.mean(pressures),
 np.std(pressures),
 np.mean(margins),
 np.std(margins),
 np.mean(temps),
 np.std(temps)
]
features.append(feature_vec)

features = np.array(features)

K-means clustering
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=self.n_clusters, random_state=42)
cluster_labels = kmeans.fit_predict(features)

Create separate Markov model for each cluster
for k in range(self.n_clusters):
 cluster_trajs = [
 traj for i, traj in enumerate(self.population_data.trajectories)
 if cluster_labels[i] == k
]
 print(f" Cluster {k}: {len(cluster_trajs)} individuals")

 # Fit cluster-specific model
 cluster_model = MarkovChainModel()
 for traj in cluster_trajs:
 for i in range(len(traj) - 1):
 state, action, reward = traj[i]
 next_state, _, _ = traj[i + 1]

 discrete_state = DiscreteState.from_continuous(state)
 discrete_next = DiscreteState.from_continuous(next_state)
 action_idx = self._discretize_action(action)

 cluster_model.observe_transition(
 discrete_state, action_idx, discrete_next, reward
)

 cluster_model.fit()
 self.cluster_models[k] = {
 'model': cluster_model,
 'centroid': kmeans.cluster_centers_[k],
 'size': len(cluster_trajs)
 }

return cluster_labels

def predict_individual_trajectory(
 self,
 initial_state: SystemState,
 actions: List[ControlInput],

```

```

horizon: int = 24,
use_cluster: Optional[int] = None
) -> List[SystemState]:
 """
 Predict trajectory for an individual using learned population model
 """
 # Choose model
 if use_cluster is not None and use_cluster in self.cluster_models:
 model = self.cluster_models[use_cluster]['model']
 else:
 model = self.markov_model

 # Simulate trajectory
 trajectory = [initial_state]
 current_discrete = DiscreteState.from_continuous(initial_state)

 for t in range(horizon):
 action = actions[t] if t < len(actions) else ControlInput()
 action_idx = self._discretize_action(action)

 # Sample next state from learned distribution
 next_discrete = model.sample_next_state(current_discrete, action_idx)

 # Convert back to continuous (approximate)
 next_state = SystemState(
 timestamp=initial_state.timestamp + timedelta(hours=t+1),
 pressure=next_discrete.pressure_bin + np.random.uniform(0, 1),
 margin=next_discrete.margin_bin + np.random.uniform(0, 1),
 temperature=next_discrete.temperature_bin + np.random.uniform(0, 1),
 drift=(next_discrete.drift_bin / 0.9) - 5 + np.random.uniform(-0.5, 0.5),
 escalation_risk=0.0 # Will be computed in __post_init__
)

 trajectory.append(next_state)
 current_discrete = next_discrete

 return trajectory

def recommend_personalized_action(
 self,
 current_state: SystemState,
 individual_cluster: Optional[int] = None
) -> Tuple[int, float]:
 """
 Recommend best action for current state
 Uses learned Q-values from population data
 """
 discrete_state = DiscreteState.from_continuous(current_state)
 s = discrete_state.to_index()

 # Choose model
 if individual_cluster is not None and individual_cluster in self.cluster_models:
 model = self.cluster_models[individual_cluster]['model']
 else:
 model = self.markov_model

 # Compute Q-value for each action: $Q(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')$

```

```

Simplified: just use immediate reward
q_values = model.reward_model[s, :]

Best action
best_action = np.argmax(q_values)
expected_value = q_values[best_action]

return best_action, expected_value

=====
LARGE NUMBER THEORY APPLICATIONS
=====

class LargeNumberTheory:
 """
 Applications of LLN, CLT, and concentration inequalities
 """

 @staticmethod
 def compute_sample_complexity(
 epsilon: float,
 delta: float,
 variance: float
) -> int:
 """
 How many samples needed to estimate mean within ϵ with probability $1-\delta$?

 Hoeffding's inequality: $P(|\bar{X} - \mu| > \epsilon) \leq 2 \exp(-2n\epsilon^2/\sigma^2)$
 Setting this $\leq \delta$ gives: $n \geq (\sigma^2 / 2\epsilon^2) \ln(2/\delta)$
 """
 n = int(np.ceil((variance / (2 * epsilon**2)) * np.log(2 / delta)))
 return n

 @staticmethod
 def concentration_bound(
 n_samples: int,
 epsilon: float,
 variance: float
) -> float:
 """
 Probability that sample mean deviates from true mean by $> \epsilon$
 Using Hoeffding's inequality
 """
 return 2 * np.exp(-2 * n_samples * epsilon**2 / variance)

 @staticmethod
 def bootstrap_confidence_interval(
 data: np.ndarray,
 statistic: Callable,
 n_bootstrap: int = 10000,
 confidence: float = 0.95
) -> Tuple[float, float]:
 """
 Bootstrap confidence interval for any statistic
 Uses resampling to approximate sampling distribution
 """

```

```

bootstrap_stats = []
n = len(data)

for _ in range(n_bootstrap):
 # Resample with replacement
 sample = np.random.choice(data, size=n, replace=True)
 bootstrap_stats.append(statistic(sample))

bootstrap_stats = np.array(bootstrap_stats)

Percentile method
alpha = 1 - confidence
lower = np.percentile(bootstrap_stats, 100 * alpha / 2)
upper = np.percentile(bootstrap_stats, 100 * (1 - alpha / 2))

return lower, upper

@staticmethod
def ergodic_theorem_verification(
 trajectory: List[float],
 stationary_mean: float,
 window_size: int = 100
) -> Dict:
 """
 Verify ergodic theorem: time average → ensemble average

 For ergodic systems: $(1/T) \sum_t f(X_t) \rightarrow E[f(X)]$ as $T \rightarrow \infty$
 """
 trajectory = np.array(trajectory)
 T = len(trajectory)

 # Compute running time average
 time_averages = []
 for t in range(window_size, T):
 time_avg = np.mean(trajectory[t-window_size:t])
 time_averages.append(time_avg)

 time_averages = np.array(time_averages)

 # Compute deviation from stationary mean
 deviations = np.abs(time_averages - stationary_mean)

 return {
 'final_time_average': time_averages[-1],
 'stationary_mean': stationary_mean,
 'mean_deviation': np.mean(deviations),
 'max_deviation': np.max(deviations),
 'convergence_rate': np.polyfit(range(len(deviations)), deviations, 1)[0]
 }

=====
ADAPTIVE INDIVIDUAL LEARNER
=====

class AdaptiveIndividualLearner:
 """

```

Learns individual-specific dynamics while leveraging population model  
Bayesian approach: prior from population, posterior from individual data  
"""

```
def __init__(
 self,
 population_learner: PopulationLearner,
 individual_id: str
):
 self.population_learner = population_learner
 self.individual_id = individual_id

 # Individual-specific data
 self.individual_trajectory = []
 self.individual_model = MarkovChainModel()

 # Bayesian combination weights
 self.population_weight = 0.7 # Initially trust population more
 self.individual_weight = 0.3

 # Assigned cluster
 self.cluster_assignment = None

def observe_individual_transition(
 self,
 state: SystemState,
 action: ControlInput,
 next_state: SystemState,
 reward: float
):
 """
 Observe transition for this specific individual
 Updates both individual and population models
 """
 self.individual_trajectory.append((state, action, reward))

 # Update individual model
 discrete_state = DiscreteState.from_continuous(state)
 discrete_next = DiscreteState.from_continuous(next_state)
 action_idx = self.population_learner._discretize_action(action)

 self.individual_model.observe_transition(
 discrete_state, action_idx, discrete_next, reward
)

 # Update population model too
 self.population_learner.ingest_individual_trajectory(
 [(state, action, reward), (next_state, ControlInput(), 0.0)],
 self.individual_id
)

 # Adapt weights as we get more individual data
 n_individual = len(self.individual_trajectory)
 self.individual_weight = min(0.7, n_individual / (n_individual + 100))
 self.population_weight = 1 - self.individual_weight

def predict_next_state_hybrid(
```

```

self,
current_state: SystemState,
action: ControlInput
) -> np.ndarray:
 """
 Hybrid prediction combining population and individual models
 Bayesian model averaging
 """
 discrete_state = DiscreteState.from_continuous(current_state)
 action_idx = self.population_learner._discretize_action(action)

 # Get predictions from both models
 if len(self.individual_trajectory) > 10:
 self.individual_model.fit()
 individual_pred = self.individual_model.predict_next_state_distribution(
 discrete_state, action_idx
)
 else:
 individual_pred = np.ones(self.individual_model.n_states) / self.individual_model.n_states

 population_pred = self.population_learner.markov_model.predict_next_state_distribution(
 discrete_state, action_idx
)

 # Weighted combination
 hybrid_pred = (
 self.population_weight * population_pred +
 self.individual_weight * individual_pred
)

 # Normalize
 hybrid_pred /= hybrid_pred.sum()

 return hybrid_pred

def assign_to_cluster(self):
 """
 Assign individual to best-matching cluster
 Uses KL divergence between individual and cluster models
 """
 if len(self.individual_trajectory) < 20:
 print(" Not enough individual data to assign cluster reliably")
 return None

 self.individual_model.fit()

 # Compute KL divergence to each cluster
 min_divergence = float('inf')
 best_cluster = None

 for cluster_id, cluster_data in self.population_learner.cluster_models.items():
 cluster_model = cluster_data['model']

 # Sample some states and compute average KL divergence
 kl_divs = []
 for _ in range(100):
 random_state = DiscreteState(

```

```

 pressure_bin=np.random.randint(0, 10),
 margin_bin=np.random.randint(0, 10),
 temperature_bin=np.random.randint(0, 10),
 drift_bin=np.random.randint(0, 10)
)
 random_action = np.random.randint(0, 10)

 p_individual = self.individual_model.predict_next_state_distribution(
 random_state, random_action
)
 p_cluster = cluster_model.predict_next_state_distribution(
 random_state, random_action
)

 # KL divergence: $\sum p \log(p/q)$
 kl = np.sum(p_individual * np.log(
 (p_individual + 1e-10) / (p_cluster + 1e-10)
))
 kl_divs.append(kl)

 avg_kl = np.mean(kl_divs)

 if avg_kl < min_divergence:
 min_divergence = avg_kl
 best_cluster = cluster_id

 self.cluster_assignment = best_cluster
 print(f" Assigned to cluster {best_cluster} (KL divergence: {min_divergence:.3f})")

 return best_cluster

def get_personalized_recommendation(
 self,
 current_state: SystemState
) -> Dict:
 """
 Get personalized action recommendation
 Uses hybrid model and assigned cluster
 """
 discrete_state = DiscreteState.from_continuous(current_state)
 s = discrete_state.to_index()

 # Use cluster model if assigned, otherwise population model
 if self.cluster_assignment is not None:
 base_model = self.population_learner.cluster_models[self.cluster_assignment]['model']
 else:
 base_model = self.population_learner.markov_model

 # Combine with individual model
 best_actions = []
 for action_idx in range(min(20, base_model.n_actions)):
 # Q-value from base model
 q_base = base_model.reward_model[s, action_idx]

 # Q-value from individual model (if enough data)
 if len(self.individual_trajectory) > 10:
 q_individual = self.individual_model.reward_model[s, action_idx]

```

```
q_hybrid = (
 self.population_weight * q_base +
 self.individual_weight * q_individual
)
else:
 q_hybrid = q_base
best
```